

Java Web Development — Understanding the Entire Stack

The biggest mistake students make is learning these topics separately.

They learn:

- Java
- JDBC
- Servlet
- JSP
- Tomcat

as five different technologies.

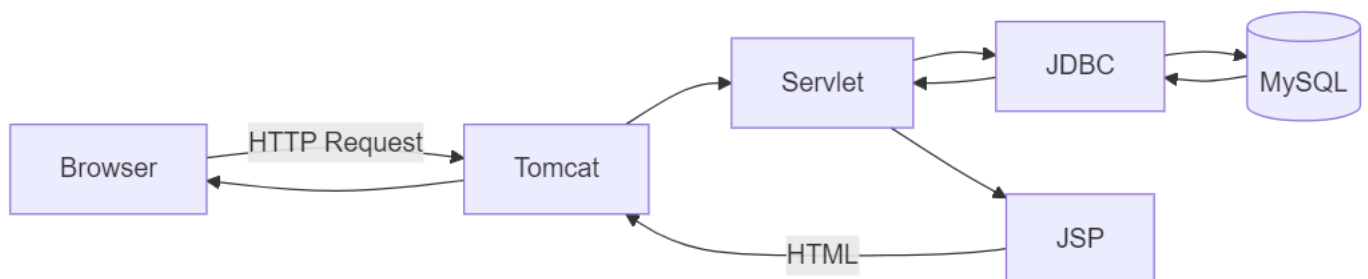
They're actually **five parts of one system**.

Think of building a restaurant.

- Java = Language used by the chefs.
- JVM = Kitchen where cooking happens.
- Tomcat = Restaurant manager.
- Servlet = Chef.
- JDBC = Waiter carrying orders to the kitchen database.
- MySQL = Pantry storing ingredients.
- JSP = Waiter decorating and serving the final dish.
- Browser = Customer.

Nothing works alone.

The Big Picture



Think of it like this.

```
1  User wants information
2
3  ↓
4
5  Browser asks server
6
7  ↓
8
9  Tomcat receives request
10
11 ↓
12
13 Servlet processes logic
14
15 ↓
16
17 JDBC fetches data
18
19 ↓
20
21 Database returns data
22
23 ↓
24
25 Servlet prepares response
26
27 ↓
28
29 JSP creates HTML
30
31 ↓
32
33 Browser displays page
```

Every Java web application follows this pattern.

Before Java Web Development

Suppose browsers could directly access databases.

```
1  Browser
2
3  ↓
4
5  Database
```

Anyone could run

```
1 DELETE FROM users;
```

No security.

So we introduced a middleman.



The server decides

- who is allowed
- what SQL runs
- what HTML returns

Java runs on the server.

What is Java?

Java is only a programming language.

Just like

Python

C++

JavaScript

It cannot receive websites by itself.

Something has to execute Java code.

JVM

The JVM is the machine that runs Java programs.

Imagine writing

```
1 System.out.println("Hello");
```

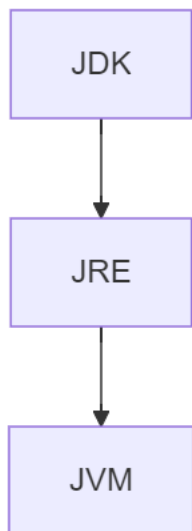
Your CPU cannot understand Java.

It only understands machine code.

So the JVM acts as a translator.



JDK vs JRE vs JVM



JVM

Runs Java.

JRE

Contains

- JVM
- Java Libraries

Enough to execute programs.

JDK

Contains

- JRE
- Compiler

- Debugger
- Development Tools

Needed for developers.

Analogy

```
1 JVM = Engine
2
3 JRE = Car
4
5 JDK = Car Factory
```

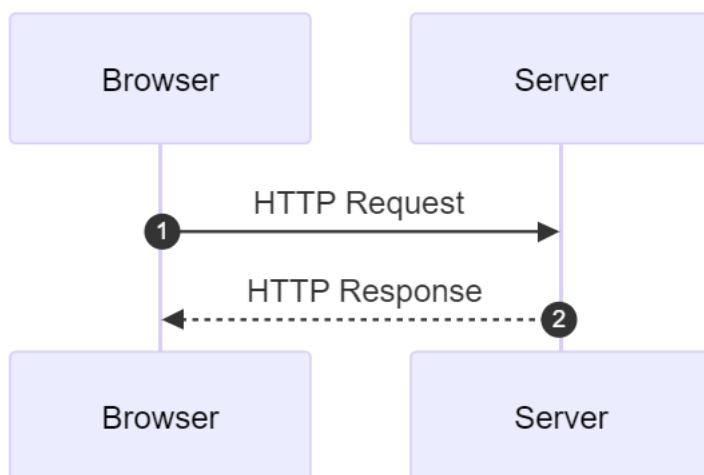
HTTP

Browsers and servers communicate using HTTP.

Example

```
1 GET /login
2
3 POST /register
4
5 GET /profile
```

Every website works by exchanging Requests and Responses.



What is Tomcat?

Tomcat is NOT your Java program.

Tomcat is a server that runs Java web applications.

Think of an apartment building.

1	Apartment Building
2	
3	↓
4	
5	Electricity
6	
7	Water
8	
9	Security
10	
11	Maintenance

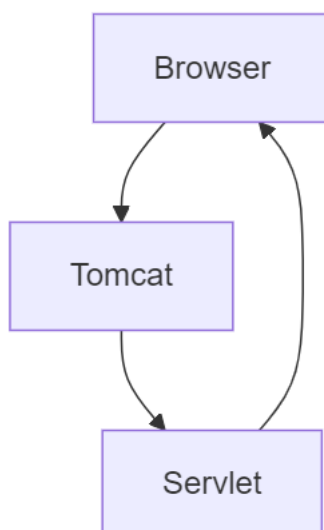
Residents only live there.

Similarly,

Tomcat provides

- HTTP handling
- Request routing
- Sessions
- Servlet management
- JSP execution

Your Java code simply lives inside Tomcat.



Tomcat is the manager.

Servlet is your employee.

What is a Servlet?

A Servlet is simply a Java class that handles HTTP requests.

If you've written Express,

```
1 app.get("/login", (req, res) => {})
```

Servlet is almost the same.

```
1 doGet(request, response)
```

Both receive

- Request
- Response

Difference is only the language.

Suppose user visits

```
1 /login
```

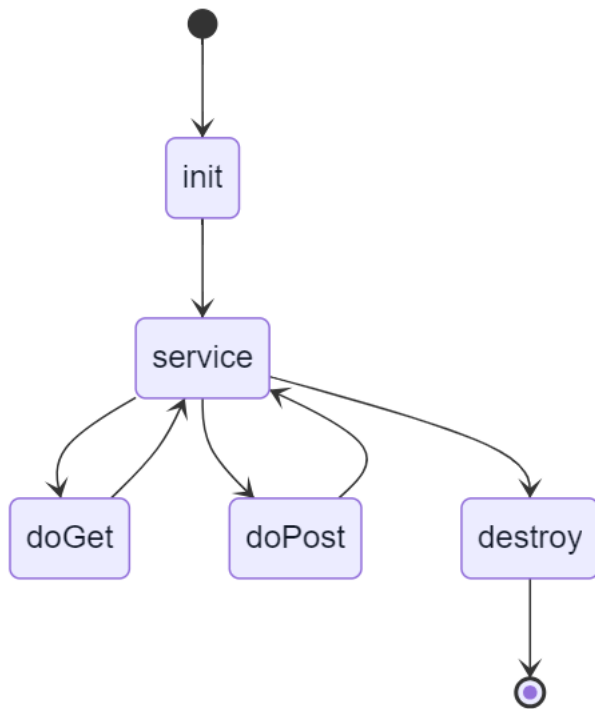
Tomcat asks

```
1 Which Servlet handles /login?
```

Then executes it.

Servlet Lifecycle

Tomcat creates a Servlet only once.



init()

Runs once.

Used for initialization.

service()

Runs for every request.

doGet()

Handles GET requests.

doPost()

Handles POST requests.

destroy()

Runs once before Tomcat removes the Servlet.

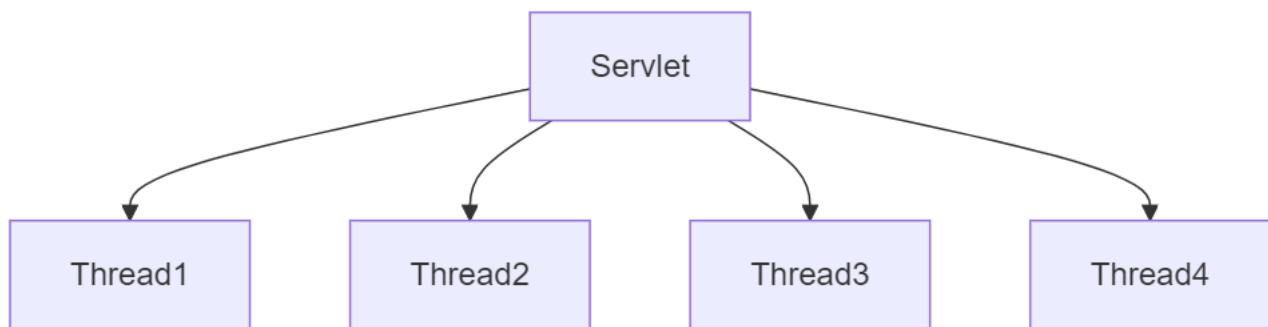
One Servlet or Thousands?

Suppose 500 students log in simultaneously.

Tomcat does NOT create 500 Servlets.

It creates one Servlet.

Then many threads execute it.



This is why Servlets should not store user-specific data in instance variables.

HttpServletRequest

Every browser request contains

- URL
- Headers
- Cookies
- Form Data

Tomcat packs everything into

```
1  HttpServletRequest
```

Example

```
1  request.getParameter("email")
```

Equivalent in Express

```
1  req.body.email
```

HttpServletResponse

Whatever you send back goes through

```
1 HttpServletResponse
```

Example

```
1 response.sendRedirect("home.jsp");
```

Equivalent

```
1 res.redirect("/home")
```

Why JDBC Exists

Java knows programming.

It doesn't know MySQL.

Every database speaks a different protocol.

JDBC creates one common API.



Driver

Think of connecting a laptop to different printers.

HP Printer

Canon Printer

Epson Printer

Each needs its own driver.

Exactly the same.

MySQL Driver

Oracle Driver

PostgreSQL Driver

All implement JDBC.

JDBC Flow



Connection

Think of making a phone call.

Without a connection,

two people cannot communicate.

Same for databases.

Statement

Used to send SQL.

```
1 SELECT *  
2 FROM STUDENT;
```

PreparedStatement

Never build SQL like this.

```
1 "..."+username+"..."
```

User might enter

```
1 ' OR 1=1 --
```

This is SQL Injection.

Instead

```
1 WHERE username=?
```

Java safely inserts values.

Almost every real project uses PreparedStatement.

ResultSet

Database returns rows.

Java stores them inside

```
1 ResultSet
```

```
1 while(rs.next())
```

Each call moves to the next row.

Why JSP Exists

Imagine creating HTML using Java.

```
1 out.println("<html>");  
2  
3 out.println("<body>");  
4  
5 out.println("<h1>");  
6  
7 out.println("Welcome");  
8  
9 out.println("</h1>");
```

This becomes impossible for large websites.

So JSP lets you write

mostly HTML

with small pieces of Java.

```
1 <h1>
2
3 Welcome <%= username %>
4
5 </h1>
```

Much cleaner.

What Actually Happens to JSP?

Many students think JSP executes directly.

It doesn't.

Tomcat secretly converts it into a Servlet.



Syntax error in text
mermaid version 11.6.0

Parse error on line 7:

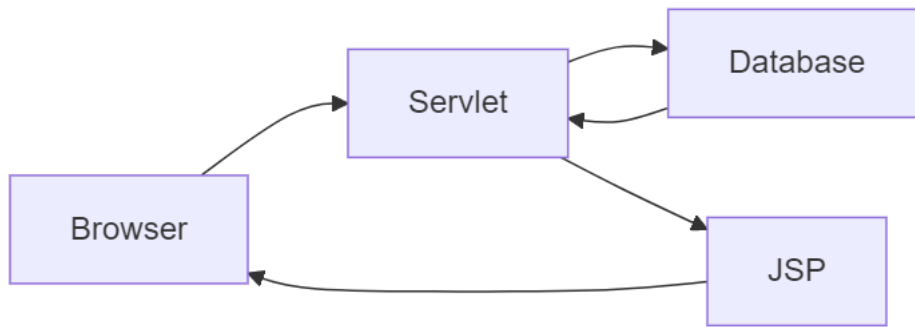
...JSPServletJava CompilerBytecode -----^ Expecting 'SEMI', 'NEWLINE', 'EOF', 'AMP', 'START_LINK', 'LINK', 'LINK_ID', got 'NODE_STRING'

Every JSP eventually becomes a Servlet.

Why Both Servlet and JSP?

Servlet is good at logic.

JSP is good at presentation.



Think of a restaurant.

Chef cooks.

Waiter serves.

Servlet = Chef

JSP = Waiter

Don't ask the waiter to cook.

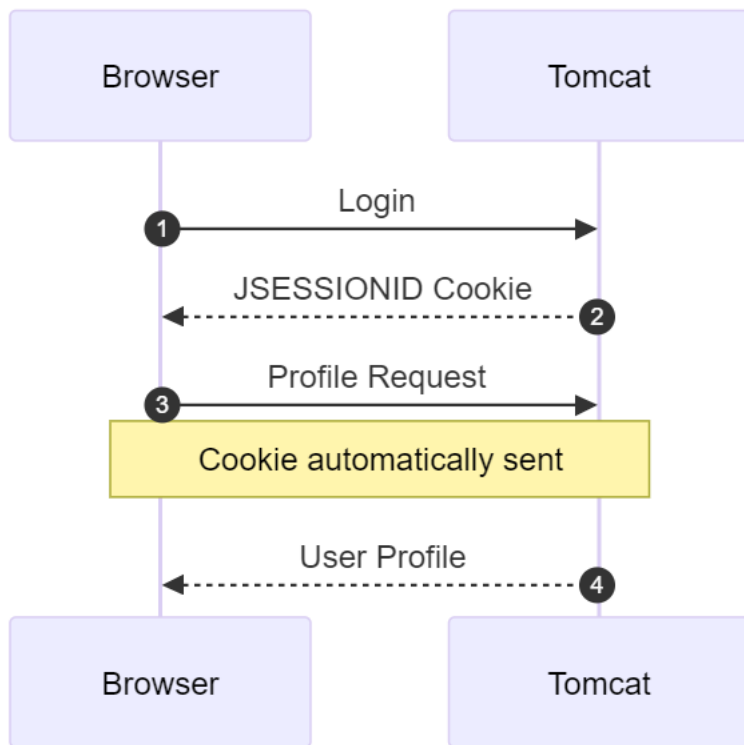
Don't ask the chef to decorate plates.

Sessions

HTTP forgets everything.

```
1  Login Request
2
3  ↓
4
5  Server replies
6
7  ↓
8
9  Next Request
10
11 ↓
12
13 Server has forgotten you
```

Sessions solve this.

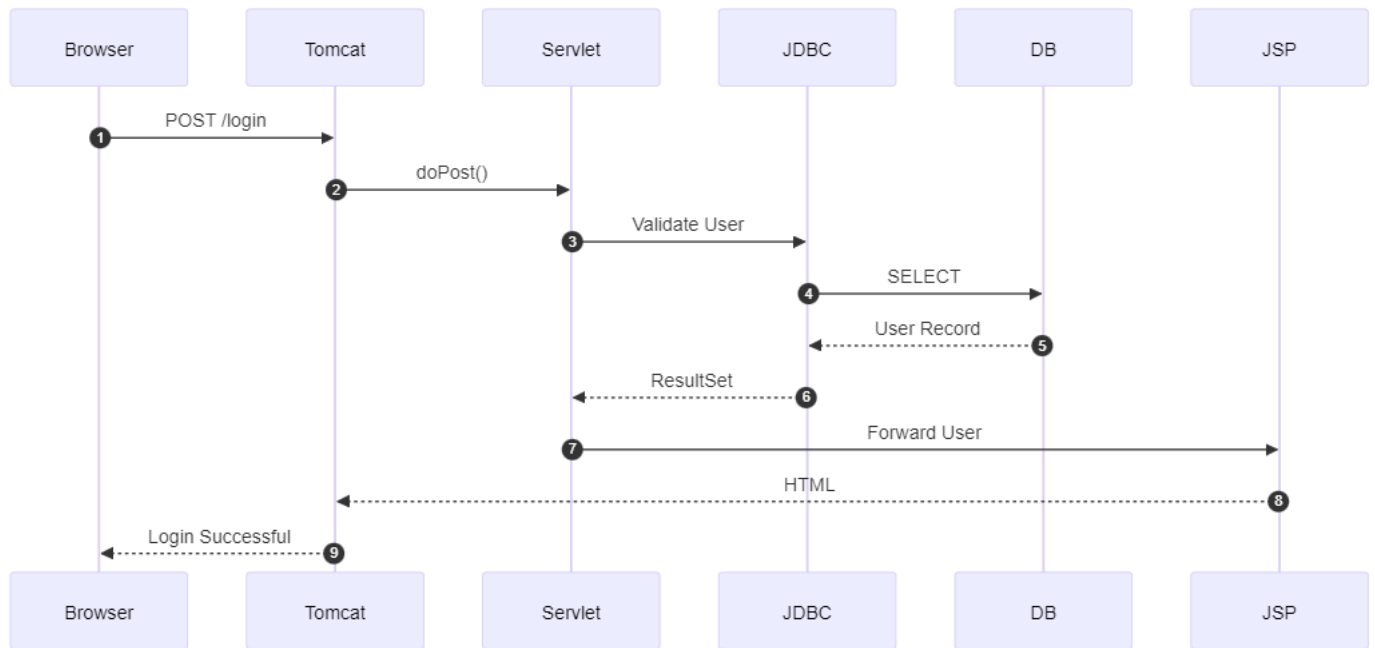


Tomcat stores

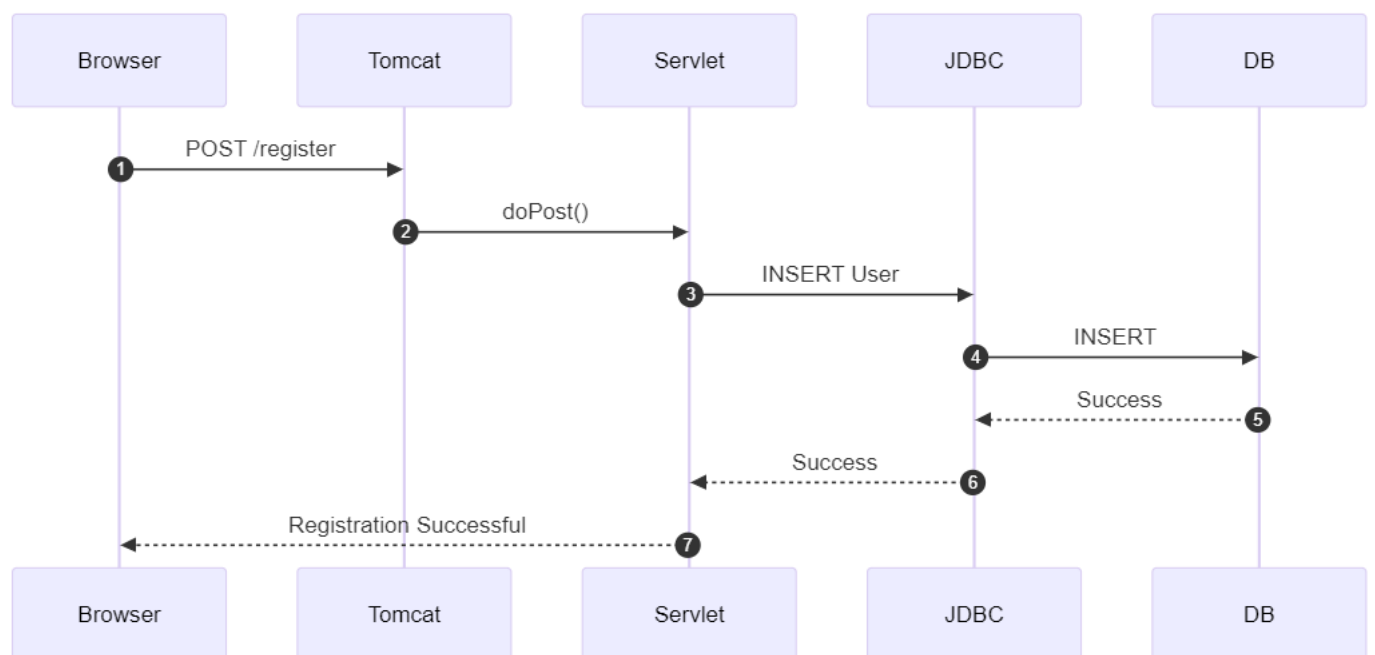
1	Session ID
2	
3	↓
4	
5	User Information

Browser stores only the Session ID.

Login Flow



Registration Flow



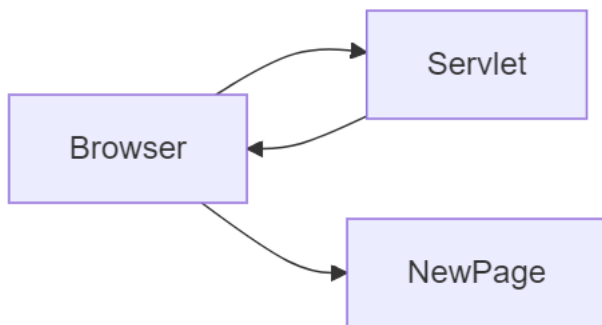
Request Forward vs Redirect

Forward



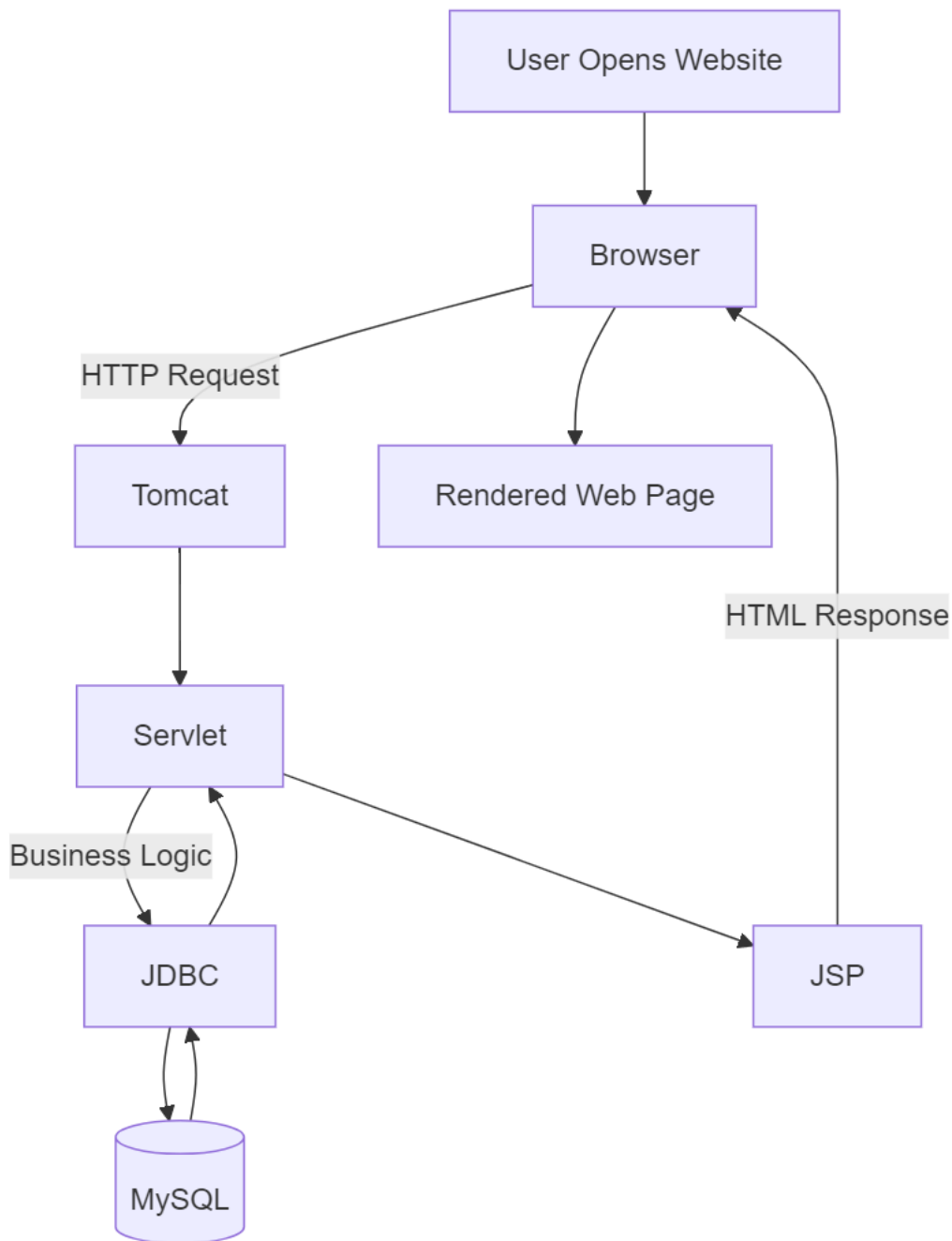
- Happens inside the server.
- Browser URL does **not** change.
- Faster.

Redirect



- Browser makes a new request.
- URL changes.
- Used after login/logout or when navigating to another resource.

Putting Everything Together



If you remember only one diagram from this guide, remember this one. It shows how the major pieces fit together: **Tomcat receives HTTP requests, Servlets execute the application logic, JDBC communicates with the database, JSP generates the HTML, and the browser displays the final page.** This is the core architecture behind traditional Java web applications.